Neuroevolution and Evolution Strategies for Reinforcement Learning

The idea of evolving neural networks rather than training them with gradient descent has a long history in artificial intelligence, one that laid essential groundwork for the modern convergence of large language models and evolutionary search. This section traces the arc from early topology-evolving algorithms through large-scale evolution strategies and, ultimately, to the evolution of entire learning algorithms, highlighting both the strengths and the persistent limitations that motivated subsequent hybrid and LLM-guided approaches.

NEAT and the Complexification Principle

Stanley and Miikkulainen introduced NeuroEvolution of Augmenting Topologies (NEAT), an algorithm that evolves both the weights and the structure of neural networks simultaneously [Stanley2002]. NEAT addressed three problems that had plagued earlier neuroevolution methods. First, it employed *historical markings*, unique innovation numbers assigned to each new gene as it arose through structural mutation, so that meaningful crossover between networks of differing topologies became possible without expensive topological analysis. Second, NEAT used *speciation* to protect structural innovations, grouping individuals by topological similarity and allowing novel structures time to optimize their weights before being forced to compete with the rest of the population. Third, NEAT started evolution from minimal networks and incrementally added complexity, a principle termed *complexification*, which biased search toward simple solutions and avoided the bloat common in methods that begin with large random topologies. On the challenging double pole-balancing benchmark without velocity information, NEAT achieved proficiency significantly faster than competing evolutionary algorithms, and the paper received the Outstanding Paper of the Decade award from the International Society for Artificial Life.

Indirect Encodings and HyperNEAT

A fundamental limitation of NEAT is that every connection in the network requires a separate gene, making the representation unwieldy for large networks. Stanley, D'Ambrosio, and Gauci addressed this with HyperNEAT, which introduced an *indirect encoding* based on Compositional Pattern-Producing Networks (CPPNs) [Stanley2009]. Rather than specifying each weight individually, a CPPN takes as input the geometric coordinates of a source and target neuron and outputs the weight of the connection between them. Because CPPNs can express spatial regularities such as symmetry, repetition, and gradient patterns, HyperNEAT can generate connectivity for networks with millions of connections from a compact genotype. The approach was demonstrated on visual discrimination tasks where networks containing over eight million connections were evolved successfully. HyperNEAT also exhibited a form of resolution independence: the same CPPN could produce connectivity patterns at different substrate resolutions without requiring further evolution, a property with no analogue in direct-encoding methods.

CoDeepNEAT and Evolving Deep Architectures

As deep learning rose to prominence, researchers adapted neuroevolution to operate at the level of architectural components rather than individual neurons. Miikkulainen et al. proposed CoDeepNEAT, a coevolutionary extension that maintained two separate populations: one of *modules*, each encoding a small subnetwork (such as a convolutional block with specific filter sizes, activation functions, and hyperparameters), and one of *blueprints*, which were graph-structured chromosomes specifying how modules should be assembled into a complete deep

network [Miikkulainen2019]. By evolving modules and blueprints in tandem, CoDeepNEAT could discover architectures competitive with hand-designed networks for image classification and language modeling, and it was applied to real-world tasks such as automated image captioning. This line of work demonstrated that evolutionary methods could scale to the design of modern deep networks, though the computational cost of evaluating each candidate architecture remained substantial.

Evolution Strategies at Scale

A pivotal moment came in 2017 when Salimans, Ho, Chen, Sidor, and Sutskever at OpenAI showed that a simple variant of evolution strategies (ES) could achieve competitive performance on standard reinforcement learning benchmarks [Salimans2017]. Their approach treated the return of a policy as a black-box objective and estimated its gradient using finite differences over a population of parameter perturbations. The critical insight was a communication trick: by sharing random seeds rather than full parameter vectors, workers needed to exchange only scalar fitness values, making the method embarrassingly parallel. With 1,440 CPU cores, the system solved the MuJoCo 3D humanoid locomotion task in approximately 10 minutes, a problem that required roughly a full day for gradient-based RL methods running on comparable hardware. On Atari games, ES achieved competitive scores after about one hour of distributed training.

The excitement around ES stemmed from its simplicity and scalability. It required no backpropagation, no value function estimation, and no replay buffer. It was naturally robust to long time horizons and sparse rewards, settings where temporal-difference methods struggle. However, the approach did not ultimately replace gradient-based RL for a fundamental reason: *sample efficiency*. ES methods require far more environment interactions than policy gradient or actor-critic algorithms to reach comparable performance, because they discard the internal structure of the trajectory and treat the entire episode return as a single scalar signal [Salimans2017]. In domains where environment interaction is expensive, such as robotics or complex simulation, this sample inefficiency proved prohibitive.

Population-Based Training

Rather than pitting evolution against gradient descent, Jaderberg et al. at DeepMind proposed Population-Based Training (PBT), a hybrid that used evolutionary mechanisms to optimize what gradient methods cannot: hyperparameters and training schedules [Jaderberg2017]. PBT maintains a population of agents, each training independently via standard gradient-based methods but with different hyperparameter configurations. Periodically, underperforming agents copy the weights of better-performing ones (exploitation) and then perturb their hyperparameters (exploration). This procedure discovers *schedules* of hyperparameter settings over the course of training rather than a single fixed configuration, capturing the intuition that the optimal learning rate or entropy coefficient may change as training progresses. PBT demonstrated improvements in deep reinforcement learning, machine translation, and generative adversarial network training, establishing that the most effective use of evolutionary ideas in deep learning might be at the meta-level rather than at the level of raw parameter optimization.

AutoML-Zero and Evolving Entire Algorithms

The most ambitious application of neuroevolution to date is AutoML-Zero, introduced by Real, Liang, So, and Le at Google Brain [Real2020]. Rather than evolving network architectures or hyperparameters, AutoML-Zero evolved complete machine learning algorithms from scratch,

starting from populations of empty programs and using only basic mathematical operations (addition, multiplication, matrix operations) as primitives. Through regularized tournament selection and mutation, the system independently rediscovered two-layer neural networks trained with backpropagation, as well as linear regression, demonstrating that evolution can navigate a search space estimated at $10^{\{12\}}$ possible programs to find needle-in-a-haystack solutions. The significance of this result was conceptual as much as practical: it established that the space of learning algorithms is itself amenable to automated search, provided the search process is sufficiently powerful.

Setting the Stage for LLM-Guided Evolution

Collectively, these works established several principles that inform the current generation of LLM-guided evolutionary methods. NEAT and HyperNEAT demonstrated that evolving structure alongside parameters produces more capable solutions than optimizing either alone. CoDeepNEAT showed that coevolution of modular components enables scaling to modern architectures. ES at scale proved that evolutionary search can be massively parallelized but also exposed the fundamental sample-efficiency gap relative to gradient methods. PBT demonstrated the power of evolving at the meta-level, optimizing the training process rather than the parameters directly. AutoML-Zero showed that evolution can discover algorithms, not merely optimize them. The natural next question, which subsequent sections of this survey address, is whether large language models, with their capacity for code generation, semantic understanding, and in-context reasoning, can serve as more intelligent mutation and crossover operators than the random perturbations used throughout the neuroevolution literature.

References

1. K. O. Stanley and R. Miikkulainen, "Evolving neural networks through augmenting topologies," *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002. DOI: 10.1162/106365602320169811.

2. K. O. Stanley, D. B. D'Ambrosio, and J. Gauci, "A hypercube-based encoding for evolving large-scale neural networks," *Artificial Life*, vol. 15, no. 2, pp. 185–212, 2009. DOI: 10.1162/artl.2009.15.2.15202.

3. R. Miikkulainen, J. Liang, E. Meyerson, A. Rawal, D. Fink, O. Francon, B. Raju, H. Shahrzad, A. Navruzyan, N. Duffy, and B. Hodjat, "Evolving deep neural networks," in *Artificial Intelligence in the Age of Neural Networks and Brain Computing*, R. Kozma, C. Alippi, Y. Choe, and F. C. Morabito, Eds. Academic Press, 2019, pp. 293–312. arXiv: 1703.00548.

4. T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, "Evolution strategies as a scalable alternative to reinforcement learning," arXiv preprint arXiv:1703.03864, 2017.

5. M. Jaderberg, V. Dalibard, S. Osindero, W. M. Czarnecki, J. Donahue, A. Razavi, O. Vinyals, T. Green, I. Dunning, K. Simonyan, C. Fernando, and K. Kavukcuoglu, "Population based training of neural networks," arXiv preprint arXiv:1711.09846, 2017.

6. E. Real, C. Liang, D. R. So, and Q. V. Le, "AutoML-Zero: Evolving machine learning algorithms from scratch," in *Proc. 37th International Conference on Machine Learning (ICML)*, 2020, pp. 8007–8019.